

- DataLeakDetector 项目介绍
 - 1. 项目结构
 - 2. 总体流水线
 - 3. 输入数据
 - 3.1 统一日志
 - 3.2 录屏视频
 - 4. 采集端
 - 4.1 Windows 采集
 - 4.2 macOS 采集
 - 5. 日志优先检测
 - 6. VLM 复核
 - 7. 抽帧策略
 - 8. OCR 与本地门控
 - 9. VLM gate 模式
 - 10. EventCorrelator
 - 10.1 输入
 - 10.2 输出
 - 10.3 前台应用分类
 - 10.4 文件 lineage
 - 10.5 UploadCandidate
 - 11. LeakReasoner
 - 11.1 Datalog 事实
 - 11.2 污点传播
 - 11.3 双引擎
 - 12. 输出报告
 - 13. NAS benchmark 结果
 - 13.1 adaptive 213643
 - 13.2 aggressive 221926
 - 13.3 adaptive 224421
 - 14. 测试覆盖
 - 15. 方案演进
 - 阶段一：视频/VLM 原型
 - 阶段二：log-first
 - 阶段三：EventCorrelator
 - 阶段四：VLM 成本和召回优化
 - 16. 当前技术取舍
 - 17. 后续改进方向
 - 18. 一句话总结

DataLeakDetector 项目介绍

DataLeakDetector 是一个桌面端数据泄露检测系统。它把操作系统日志、窗口上下文、文件操作、剪贴板、录屏画面、OCR/VLM 视觉理解和 Datalog 符号推理组合起来，判断敏感文件或敏感内容是否经过复制、派生、上传、发送、共享屏幕、截图、远程桌面等路径离开可信环境。

这个项目的核心思路不是“看到敏感文件就报警”，也不是“把整段视频交给大模型”。它的目标是构造一条可审计的证据链：

敏感源头

-> 文件/内容派生

-> 应用或进程间传播

-> 外部 sink 暴露

-> 可解释的泄露路径

系统最终要回答四个问题：

问题	示例
哪个敏感对象被涉及	工资表.xlsx、合同.docx、截图中的表格内容、剪贴板文本
它经历了什么变化	复制、压缩、转换 PDF、重命名、截图、粘贴到输入框
它进入了哪里	邮箱附件、聊天窗口、AI 服务、云盘、会议共享、U 盘、远程桌面
判断依据是什么	日志事件、窗口标题、文件 lineage、关键帧、OCR 文本、VLM verdict、Datalog 路径

1. 项目结构

当前仓库按“采集工具 + 三段检测主链路 + 评测材料”组织：

DataLeakDetector/

├─ tools/ScreenMonitor/

├─ 01-FrameAnalyzer/

├─ 02-EventCorrelator/

├─ 03-LeakReasoner/

├─ main/

├─ spec/

└─ tests/

Windows/macOS 日志与录屏采集

视频抽帧、OCR、VLM 分析，以及历史 RiskHunter 逻辑

日志、窗口、文件 lineage、视觉片段的证据关联

Datalog 污点传播和泄露路径推理

统一包入口和端到端编排

架构说明、实验报告、fixtures、benchmark 输出

回归测试

推荐的新代码导入路径是 `main/data_leak_detector` 里的统一包，而不是直接依赖历史目录：

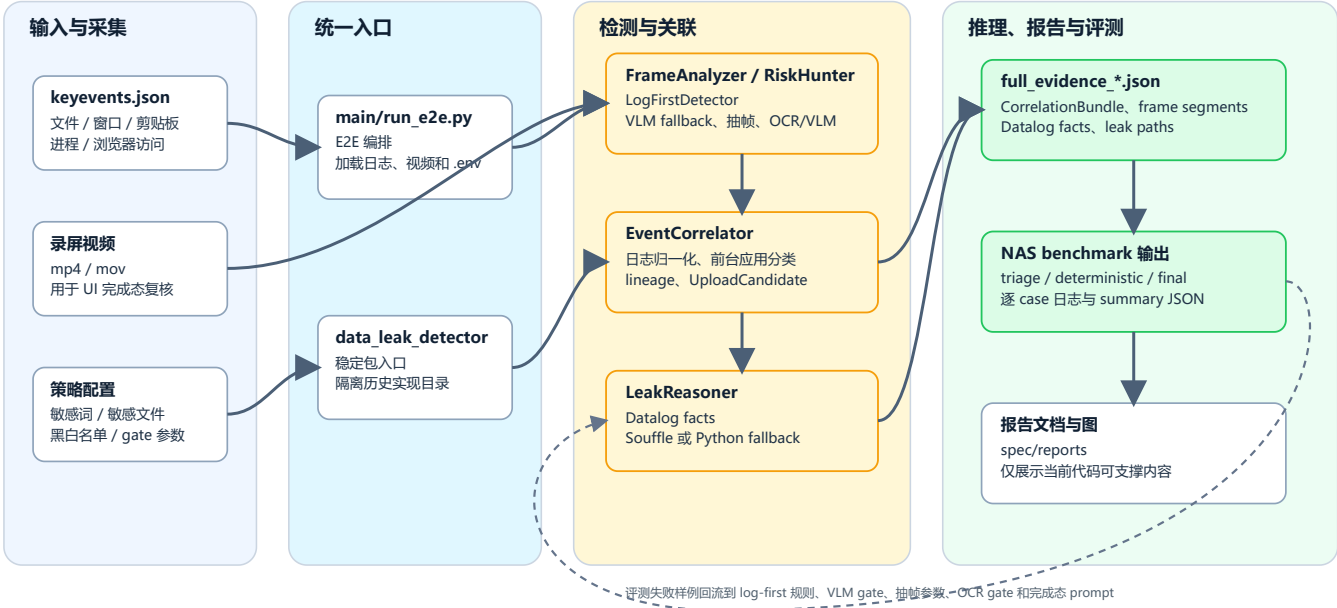
```
from data_leak_detector.event_correlator import EventCorrelator
from data_leak_detector.frame_analyzer import analyze_video_behavior
from data_leak_detector.leak_reasoner import DatalogEngine
```

历史路径统一在 `main/data_leak_detector/legacy_paths.py` 中维护。这样做的原因很实际：项目里已经有大量脚本依赖旧目录名，直接重命名会破坏 benchmark 和 E2E；统一入口可以先稳定 API，再逐步迁移物理目录。

2. 总体流水线

DataLeakDetector 当前实现架构

以 main/run_e2e.py 和 tools/benchmark_nas_samples.py 为准：日志优先、视觉复核、证据关联、Datalog 推理



完整检测链路可以概括为：

keyevents.json + video.mp4

- > 采集日志归一化
- > 日志优先检测
- > 可疑时间窗构建
- > 抽帧/OCR/VLM 复核
- > 事件关联与文件 lineage
- > Datalog 泄露路径推理
- > JSON 证据报告

端到端入口是 main/run_e2e.py：

```
python main/run_e2e.py --log path\to\keyevents.json --video path\to\recording.mp4
```

运行时主要经历四个阶段：

阶段	代码位置	作用
加载输入	main/run_e2e.py	读取日志、录屏路径、录屏开始时间
FrameAnalyzer / RiskHunter	01-FrameAnalyzer/	先做 log-first 检测；必要时抽帧并调用 VLM
EventCorrelator	02-EventCorrelator/	把日志、窗口、VLM segment、文件派生链合成结构化证据
LeakReasoner	03-LeakReasoner/	将证据转成 Datalog fact，推理泄露路径

最终报告写到 spec/output/full_evidence_*.json。

3. 输入数据

3.1 统一日志

采集端最终给检测链路的核心输入是 `keyevents.json`。它是一个 JSON Array，每个元素是一条关键事件：

```
{
  "timestamp": "2026-02-11T12:27:36.000",
  "event_type": "created",
  "file_path": "D:\\work\\AAA公司服务合作合同.docx",
  "file_name": "AAA公司服务合作合同.docx",
  "file_extension": ".docx",
  "process_info": {
    "pid": "11488",
    "process_name": "chrome.exe",
    "process_path": ""
  },
  "window_info": {
    "window_title": "打开",
    "window_class": "#32770"
  },
  "app_name": "Chrome",
  "extra": {
    "raw_operation": "browser_file_access",
    "source": "etw_monitor"
  }
}
```

重要字段：

字段	用途
timestamp	和录屏帧、窗口、文件操作对齐
event_type	判断打开、复制、上传、剪贴板、窗口切换等动作类型
file_path / file_name	识别敏感文件和派生文件
process_info	识别行为进程
window_info.window_title	判断邮箱、AI 服务、网盘、文件选择框、会议页面等前台上下文
extra.source	区分 watchdog、ETW、fs_usage、Unified Logs、clipboard monitor 等来源

常见 event_type：

类型	含义
opened	文件被打开或读取
created / modified / renamed / deleted	文件系统变化
clipboard_text / clipboard_image	剪贴板文本或图片
app_switch	前台窗口变化

类型	含义
<code>file_selected</code>	文件选择框选择文件
<code>upload_detected</code>	启发式上传事件
<code>manual_note</code>	人工补充事件

3.2 录屏视频

录屏视频用于补足日志看不到的信息。例如：

- 页面上是否出现“发送成功”。
- 敏感内容是否已经粘贴到 AI/chat 输入框。
- 会议共享时敏感文件是否可见。
- 远程桌面/虚拟机里是否完成复制或上传。
- 邮箱附件只是被选择，还是已经发送。

项目不会默认全量分析整段视频，而是先由日志限定可疑窗口，再在窗口内抽取代表帧。

4. 采集端

采集端在 `tools/ScreenMonitor`。

4.1 Windows 采集

Windows 端位于 `tools/ScreenMonitor/windows_monitor`，主要能力包括：

- 使用 Web UI 创建和停止监控会话。
- 录制屏幕。
- 使用 watchdog 捕获文件创建、修改、删除、重命名。
- 使用 C++ ETW 捕获浏览器进程读取本地文件的行为，这对“网页上传文件”很关键。
- 捕获剪贴板文本和图片。
- 捕获窗口切换和窗口标题。
- 输出 `logs.json` 和下游使用的 `keyevents.json`。

ETW 需要管理员权限。浏览器上传场景下，普通文件系统监控未必能准确知道“哪个网页读取了哪个文件”，ETW 可以提供更强证据。

4.2 macOS 采集

macOS 端位于 `tools/ScreenMonitor/Mac_monitor`，主要能力包括：

- Go 后端管理会话和 API。
- React/Vite 前端展示会话、视频和日志。
- FFmpeg 录屏。

- FSEvents 捕获文件系统变化。
- `fs_usage` 捕获文件打开/读取。
- Unified Logs 捕获文件选择对话框、系统分享、AirDrop 等高级事件。

macOS 端需要授予屏幕录制、辅助功能和完全磁盘访问权限。

5. 日志优先检测

项目当前非常依赖 log-first 思路。它的基本判断是：

如果日志已经足够说明敏感文件被外发，就不应该花钱调用 VLM；如果日志只能说明“有可疑上下文”，才让 VLM 看关键帧。

核心代码是 `01-FrameAnalyzer/risk_hunter/log_first_detector.py`。

LogFirstDetector 做的事情包括：

1. 识别敏感文件。
 - 配置中的敏感文件路径。
 - 文件名中的敏感词，例如合同、机密、工资、财务、密码。
 - 剪贴板内容、窗口标题、内容预览中的敏感词。
2. 过滤噪声。
 - 系统缓存路径。
 - 临时文件。
 - 无关应用或白名单路径。
3. 建立文件映射。
 - 原文件到复制文件。
 - 原文件到重命名文件。
 - 原文件到压缩包。
 - 原文件到转换产物，例如 docx -> pdf。
4. 判断显式外发。
 - 上传。
 - 发送。
 - 邮件附件。
 - 聊天文件发送。
 - 网盘同步。
 - 截图或剪贴板内容进入外部应用。
5. 输出检测结果。
 - `upload_events`
 - `operation_records`

- file_mappings
- vlm_fallback 元数据
- statistics

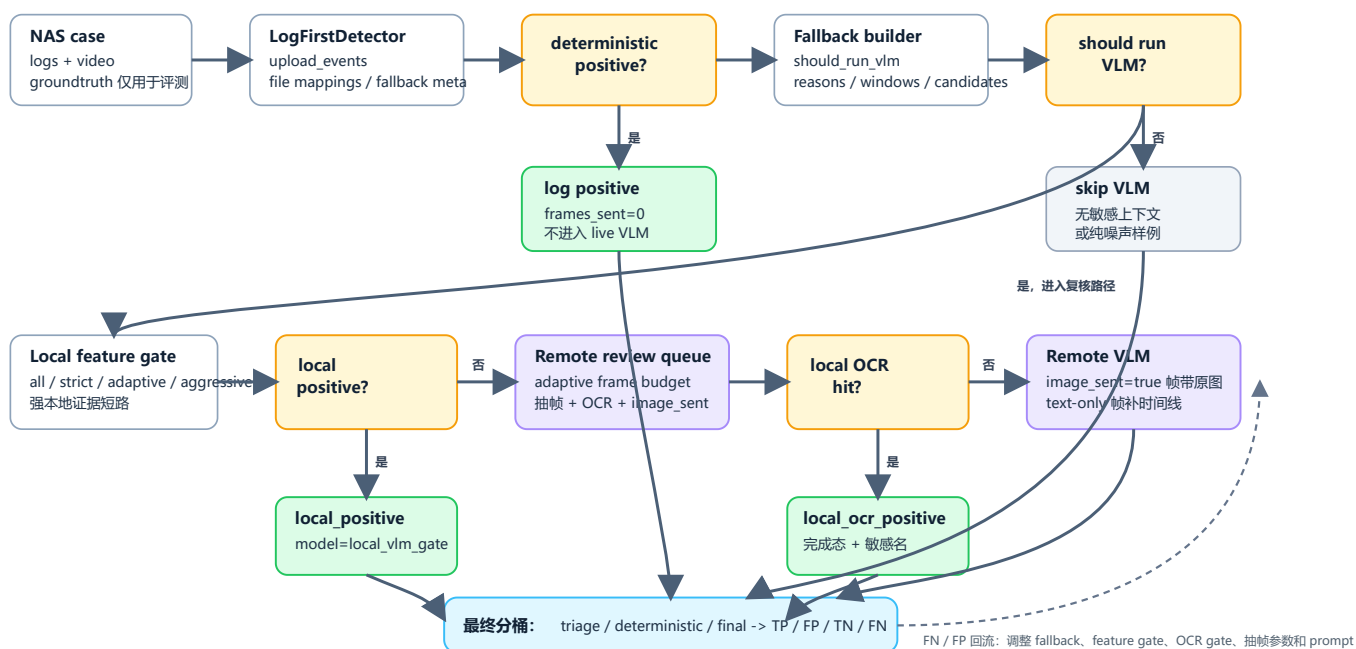
如果 log-first 已经发现可靠上传链路，E2E 会跳过 VLM 阶段，直接把结果交给 EventCorrelator 和 LeakReasoner。

6. VLM 复核

VLM 只处理日志无法确定但值得复核的样例。触发逻辑主要来自 `suspicious_window_builder.py` 和 `tools/benchmark_nas_samples.py`。

VLM 复核门控流程

当前实现：先确定性日志判断，再 local feature gate，最后才进入 OCR / remote VLM 复核



典型需要 VLM 的场景：

- 敏感文件附近出现 AI 服务，但日志不知道是否粘贴了内容。
- 文件被选择为附件，但日志不知道是否点击发送。
- 会议或屏幕共享开始，但日志不知道画面里是否显示敏感内容。
- 远程桌面或虚拟机中发生复制/上传，宿主机日志不完整。
- 出现截图、剪贴板图片、OCR 提取等内容级泄露迹象。

典型不需要 VLM 的场景：

- 日志已经明确捕获敏感文件上传到黑名单应用。
- 没有任何敏感文件或敏感文本上下文。
- 只是普通浏览或普通聊天，和敏感 anchor 距离很远。

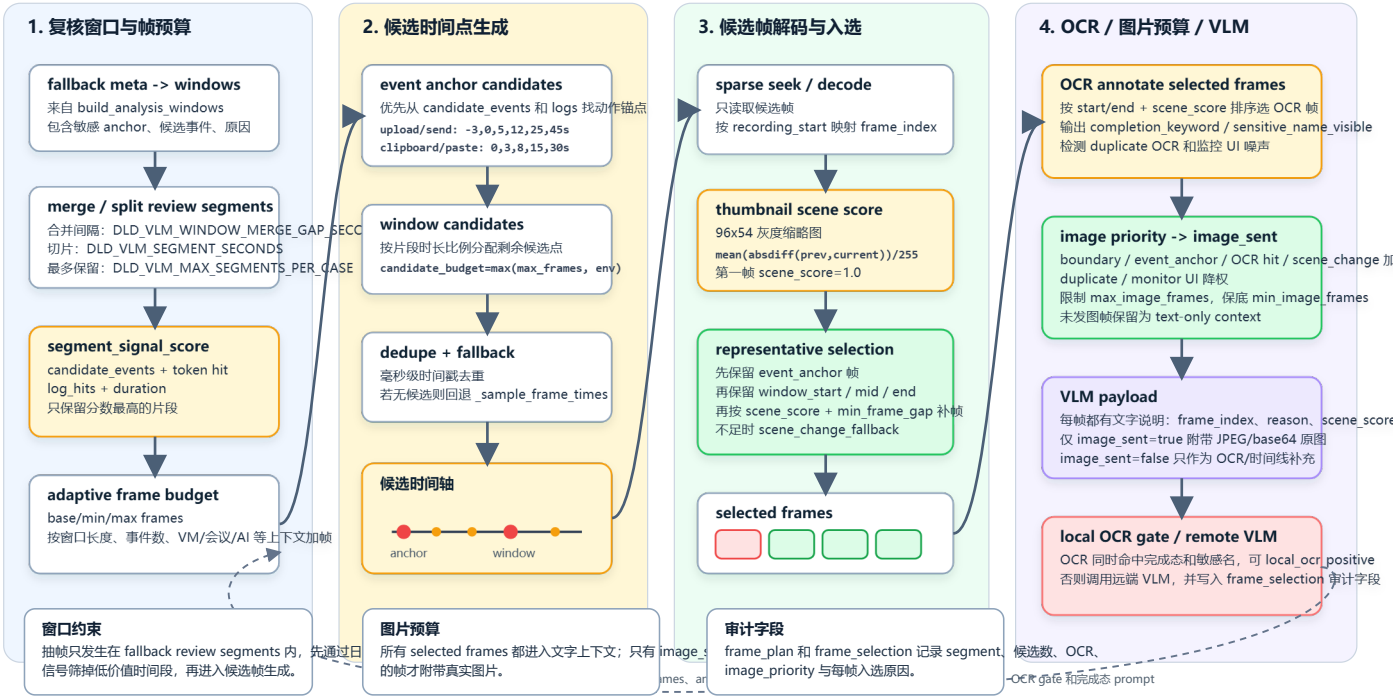
7. 抽帧策略

早期做法是在可疑窗口内均匀取几帧。这很容易错过短暂 UI 状态，例如“发送成功”提示只出现一两秒。

当前策略更准确地说是“高信号片段选择 + 事件锚点候选 + 场景变化评分 + OCR/图片预算二次筛选”：

当前 NAS live VLM 抽帧流程

对应 tools/benchmark_nas_samples.py: 先选高信号复核片段，再用事件锚点、场景变化、OCR 和图片预算生成 VLM payload



对应当前代码路径: _adaptive_vlm_frame_budget -> _prepare_review_segments -> _frame_time_candidates -> _select_representative_frames -> _annotate_and_limit_image_frames -> _live_vlm_review_case。

当前代码中的实际步骤：

1. 从 `fallback_meta` 和日志得到 VLM review windows。
2. 先对窗口做 merge，再按 `DLD_VLM_SEGMENT_SECONDS` 切成 review segments，并用 candidate event、日志 token 命中和时长给 segment 打分，只保留最高信号的几个片段。
3. 根据窗口数量、总时长、候选事件数、VM/远程桌面、会议、剪贴板、上传、AI 等上下文计算自适应帧预算。
4. 先从候选事件和日志中提取 `event_anchor_*` 时间点，例如上传/发送事件附近的 -3, 0, 5, 12, 25, 45s，剪贴板/粘贴事件附近的 0, 3, 8, 15, 30s。
5. 再按 segment 时长比例补充普通 `window_candidate` 时间点，默认候选预算来自 `DLD_VLM_REVIEW_CANDIDATE_FRAMES`，常见值为 `max(24, max_frames * 6)`。
6. 对候选时间点去重，然后只 seek/decode 这些候选帧，不全量扫描视频。
7. 将候选帧缩放为 96x54 灰度缩略图，计算相邻候选帧差异，得到 `scene_score`。
8. 代表帧选择时优先保留 event anchor，再保留窗口开头、中间、结尾，最后按 `scene_score` 和 `DLD_VLM_REVIEW_MIN_FRAME_GAP` 补入 scene-change 帧。
9. 对入选帧运行受限数量的 OCR，生成 `completion_keyword`、`sensitive_name_visible`、`ocr_duplicate` 等标记。
10. 根据 boundary、event anchor、OCR hit、scene change、重复 OCR、监控 UI 噪声等计算 `image_priority`，决定哪些帧真正 `image_sent=true` 附带 JPEG/base64 原图。

- 11. 所有 selected frames 都进入 VLM 文字上下文；只有 image_sent=true 的帧发送真实图片，image_sent=false 的帧只提供 OCR/时间线补充。
- 12. frame_plan 和 frame_selection[] 会记录 segment 计划、候选数、OCR、图片决策和每帧入选原因，便于复查。

抽帧记录示例：

```
{
  "frame_index": 128,
  "timestamp": "2026-06-03 00:41:51",
  "selection_reason": "scene_change",
  "scene_score": 0.42,
  "ocr_text": "上传完成",
  "ocr_flags": ["completion_keyword"],
  "ocr_ran": true,
  "image_priority": 6.42,
  "image_decision_reasons": ["event_anchor", "ocr_risk_hit", "scene_change"],
  "image_sent": true
}
```

这样后续排查误报/漏报时可以知道：哪些窗口片段被保留、候选帧来自事件锚点还是窗口采样、哪些帧只是文本上下文、哪些帧真的发给了 VLM，以及 OCR/图片预算是否导致关键完成态被降权。

8. OCR 与本地门控

OCR 的作用不是替代 VLM，而是在调用远端模型前减少无意义图片和请求。

当前支持：

OCR engine	说明
easyocr	中英文 OCR，GPU 可用时使用 CUDA
rapidocr	基于 ONNX Runtime，适合轻量部署
none	关闭 OCR
auto	根据环境选择

相关环境变量：

变量	含义
DLD_VLM_OCR_ENGINE	选择 easyocr、rapidocr、none、auto
DLD_VLM_ENABLE_OCR_PREFILTER	是否启用 OCR 预筛
DLD_VLM_LOCAL_OCR_GATE	OCR 命中完成态和敏感名时是否本地 positive
DLD_VLM_REVIEW_MAX_OCR_FRAMES	每个 case 最多 OCR 几帧

OCR 风险标记：

标记	含义
completion_keyword	看到发送成功、上传完成、已分享等完成态
preliminary_keyword	看到附件、上传中、选择文件等准备态
sensitive_name_visible	看到敏感文件名或敏感关键词

如果同一批关键帧里同时出现 completion_keyword 和 sensitive_name_visible，系统可以生成 local_ocr_positive，不再调用远端 VLM。

9. VLM gate 模式

NAS benchmark 支持多种 VLM gate，用来控制远端请求量：

模式	行为
all	所有 triage-only case 都进入远端 VLM
strict	只有很强的本地证据才直接 positive
adaptive	在 strict 基础上加入 VM/远程桌面等高置信上下文
aggressive	进一步允许 Git、压缩/转换等本地完成态场景直接 positive

这些 gate 只看运行时证据，不看样本名、groundtruth 或 case 顺序。

常用 benchmark 命令：

```
$ts = Get-Date -Format "yyyyMMdd_HH:mm:ss"
$log = "spec\output\nas_vlm_adaptive_$ts.log"
$json = "spec\output\nas_vlm_adaptive_$ts.json"

$env:DLD_VLM_REVIEW_MAX_OCR_FRAMES = "2"
$env:DLD_VLM_REVIEW_MAX_IMAGE_FRAMES = "4"
$env:DLD_VLM_REVIEW_CACHE = "1"
$env:DLD_VLM_LOCAL_OCR_GATE = "1"

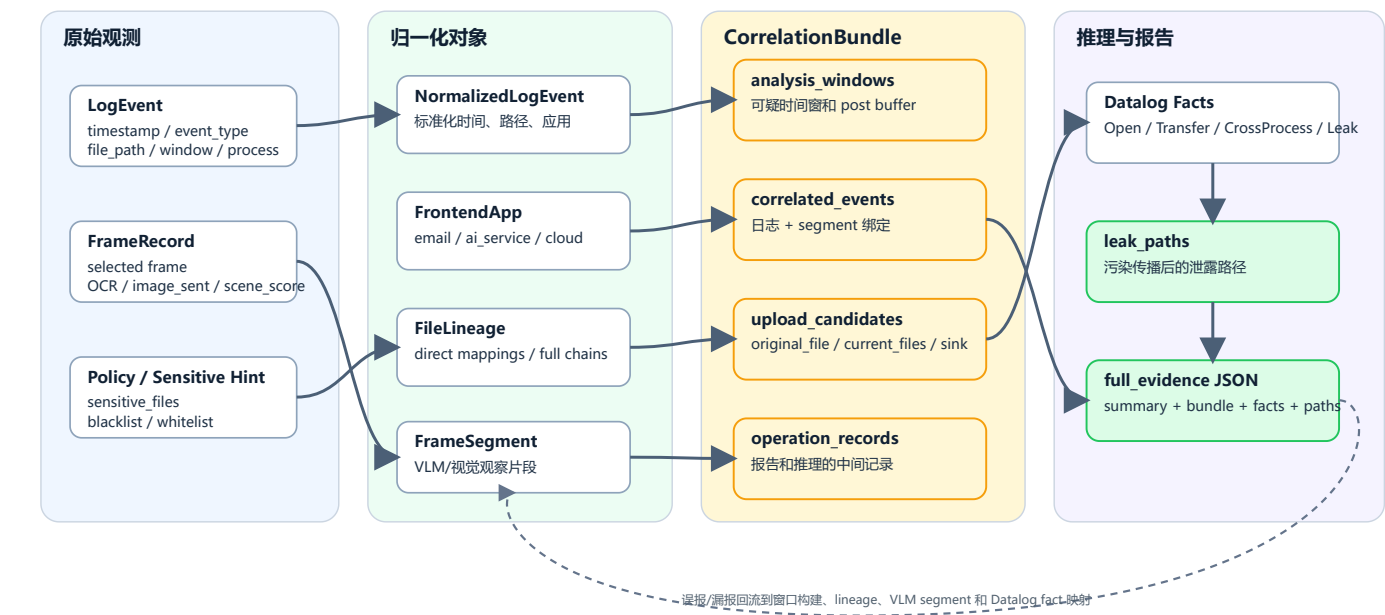
python tools\benchmark_nas_samples.py `
    --use-vm `
    --vlm-gate-mode adaptive `
    --vlm-workers 4 `
    --max-vm-frames 10 `
    --json-output $json `
    2>&1 | Tee-Object -FilePath $log
```

10. EventCorrelator

EventCorrelator 是项目里最关键的“证据整理层”。它不直接决定最终是否泄露，而是把上游零散材料整理成下游可推理的结构。

当前证据结构

不是已落地的 Neo4j 图；当前实现主要落在 CorrelationBundle、file_lineage、UploadCandidate、Datalog facts 和 JSON 报告中



说明：Neo4j 写入仍属于后续计划；本图只描述当前代码已经输出或消费的数据结构。

10.1 输入

EventCorrelatorInput 包括：

```
{
  "session_id": "...",
  "record_id": "...",
  "log_events": [...],
  "frame_segments": [...],
  "sensitive_files": [...],
  "recording_start_time": "...",
  "session_metadata": {...},
  "correlation_config": {...}
}
```

其中 frame_segments 来自 VLM 或历史 FrameAnalyzer 输出，包含：

- 时间范围。
- 应用名。
- 操作类型。
- 主资源和相关资源。
- 可见证据。
- 置信度。

10.2 输出

CorrelationBundle 包含：

字段	说明
<code>analysis_windows</code>	建议后续视觉分析的时间窗
<code>correlated_events</code>	日志和视觉片段绑定后的事件
<code>operation_records</code>	面向报告和推理的操作记录
<code>upload_candidates</code>	可能的外发候选
<code>file_lineage</code>	文件派生关系
<code>statistics</code>	输入输出数量和中间统计
<code>errors</code>	可恢复错误

10.3 前台应用分类

浏览器进程名太粗糙，`chrome.exe` 可能是邮箱、AI 服务、网盘、会议或普通网页。EventCorrelator 会根据窗口标题和 URL 进一步归类为：

- `email`
- `ai_service`
- `cloud_storage`
- `meeting`
- `chat`
- `code_repo`
- `browser`
- `unknown`

这个分类会影响后续 `sink_type` 判断。

10.4 文件 lineage

文件泄露经常不是原文件直接上传，而是先派生：

```
客户名单.xlsx -> 客户名单.pdf -> 客户名单.zip -> 网页上传
```

`lineage.py` 会根据以下信息建立映射：

- 显式 `parent` 字段。
- `related_paths`。
- 创建、重命名、复制、压缩、转换事件。
- 已知敏感 artifact 的文件名 stem 推断。

导出结构：

```
{
  "direct_file_mappings": {
    "c:/work/customer.zip": "c:/work/customer.pdf"
```

```

},
"full_file_mapping_chains": {
  "c:/work/customer.zip": "c:/work/customer.xlsx -> c:/work/customer.pdf -> c:/work/customer.zip"
},
"artifact_instances": [
  {
    "artifact_id": "c:/work/customer.zip@2026-01-01 10:00:20",
    "path": "c:/work/customer.zip",
    "parent_path": "c:/work/customer.pdf",
    "root_path": "c:/work/customer.xlsx"
  }
]
}

```

10.5 UploadCandidate

UploadCandidate 是 EventCorrelator 给下游的核心风险候选：

```

{
  "candidate_id": "upload_12",
  "original_file": "c:/work/customer.xlsx",
  "current_files": ["c:/work/customer.zip"],
  "app_name": "Chrome",
  "operation_type": "upload",
  "sink_type": "web_post",
  "evidence_refs": ["log:evt_10", "segment:seg_2"],
  "mapping_links": [
    "c:/work/customer.xlsx -> c:/work/customer.pdf -> c:/work/customer.zip"
  ],
  "confidence": 0.91,
  "object_binding": {
    "binding_type": "lineage",
    "binding_confidence": 0.95,
    "bound_asset": "c:/work/customer.xlsx"
  }
}

```

支持的 sink 类型包括：

sink	场景
mail_attachment	邮件附件
chat_upload	IM/聊天发送
cloud_sync	网盘同步或上传
screen_share	会议共享或屏幕暴露
web_post	普通网页上传

11. LeakReasoner

LeakReasoner 位于 **03-LeakReasoner**，负责把上游证据转成符号事实并推理泄露路径。

核心 Datalog 规则文件：**03-LeakReasoner/datalog/taint_rules.dl**

11.1 Datalog 事实

主要关系：

```
OpenFile(id, process, file, timestamp)
TransferFile(id, process, src, dst, timestamp)
CrossProcessTransfer(id, from_process, to_process, shared_data, timestamp)
LeakFile(id, process, file, leak_channel, timestamp)
ClipboardWrite(id, process, data, timestamp)
ClipboardRead(id, process, data, timestamp)
```

这些关系可以表达：

```
Excel 打开 工资表.xlsx
Excel 将内容写入 Clipboard
WeChat 从 Clipboard 读取内容
WeChat 通过 network 发送
```

11.2 污点传播

推理逻辑：

- 1. **OpenFile** 将敏感文件标为污点源。
- 2. **TransferFile** 传播同进程内的数据变化。
- 3. **CrossProcessTransfer** 传播跨进程数据。
- 4. **ClipboardWrite** + **ClipboardRead** 可以在 5 分钟窗口内归纳为跨进程传播。
- 5. **LeakFile** 如果作用在污点数据上，则输出 **SearchLeak**。

输出泄露路径示例：

```
OpenFile(op_1, Excel, 工资表.xlsx)
-> TransferFile(op_2, Excel, 工资表.xlsx, Clipboard)
-> CrossProcessTransfer(op_3, Excel, WeChat, Clipboard)
-> LeakFile(op_4, WeChat, Clipboard, network)
```

11.3 双引擎

项目支持两种 Datalog 引擎：

引擎	场景
Souffle	Linux/macOS 上安装 Souffle 时使用，性能更好
Python fallback	Windows 或未安装 Souffle 时使用，零额外依赖

运行时自动检测 **souffle** 命令。未找到时会打印 warning，然后切换到 Python 实现。

12. 输出报告

E2E 报告是 JSON。需要先说明一点：`main/run_e2e.py` 的报告字段仍保留部分历史编号，例如 `module3_risk_hunter`、`module4_threat_detector`、`module4_datalog_facts`。这些字段名是兼容旧脚本的结果，不代表当前推荐的新模块命名。

当前实际运行顺序是：

```
FrameAnalyzer / RiskHunter -> EventCorrelator -> LeakReasoner
```

报告主要字段包括：

```
{
  "report_id": "full_evidence_20260701_120000",
  "generated_at": "...",
  "input": {
    "log_file": "...",
    "video_file": "..."
  },
  "summary": {
    "module1_event_correlator_windows": 3,
    "module1_event_correlator_upload_candidates": 1,
    "module2_frame_analyzer_observations": 4,
    "module4_datalog_facts": 6,
    "module4_leak_paths": 1
  },
  "target_three_module_architecture": {
    "module1_event_correlator": {},
    "module2_frame_analyzer": {},
    "module3_leak_reasoner": {}
  },
  "conclusion": "发现数据泄露风险"
}
```

上面数字是结构示例，不是固定评测结果；真实值以每次 `full_evidence_*.json` 的 `summary` 为准。

13. NAS benchmark 结果

来源文件是：

- `spec/output/nas_vlm_adaptive_20260630_224421.json`
- `spec/output/nas_vlm_adaptive_20260630_213643.json`
- `spec/output/nas_vlm_aggressive_20260630_221926.json`

三组完整跑完的配置和耗时如下。日志没有给每一条 VLM 事件写统一时间戳，所以总耗时按 log 文件创建时间到最后写入时间估算；远端 VLM 阶段按第一条 VLM 队列/调用记录到最后写入时间近似。

run	gate	workers	文件时间 窗口	总耗时	远端 VLM 阶 段近似	remote / local / live
nas_vlm_adaptive_20260630_213643	adaptive	4	21:36:46- 22:13:01	36m15s	33m55s	124 / 42 / 166
nas_vlm_aggressive_20260630_221926	aggressive	6	22:19:28- 22:42:08	22m39s	20m03s	60 / 106 / 166
nas_vlm_adaptive_20260630_224421	adaptive	4	22:44:25- 23:40:23	55m58s	55m58s	100 / 66 / 166

13.1 adaptive 213643

指标	Precision	Recall	F1	TP	FP	TN	FN
triage	87.88%	100.00%	93.55%	203	28	16	0
deterministic	96.92%	31.03%	47.01%	63	2	42	140
final	93.85%	60.10%	73.27%	122	8	36	81

统计：

total cases: 247
deterministic hits: 65
vlm reviews: 231
live vlm reviews: 166
remote vlm requests: 124
local vlm resolutions: 42
skipped cases: 16

13.2 aggressive 221926

指标	Precision	Recall	F1	TP	FP	TN	FN
triage	87.88%	100.00%	93.55%	203	28	16	0
deterministic	96.92%	31.03%	47.01%	63	2	42	140
final	92.27%	82.27%	86.98%	167	14	30	36

统计：

total cases: 247
deterministic hits: 65
vlm reviews: 231
live vlm reviews: 166


```
remote vlm requests: 60
local vlm resolutions: 106
skipped cases: 16
```

13.3 adaptive 224421

指标	Precision	Recall	F1	TP	FP	TN	FN
triage	87.88%	100.00%	93.55%	203	28	16	0
deterministic	96.92%	31.03%	47.01%	63	2	42	140
final	90.52%	94.09%	92.27%	191	20	24	12

统计：

```
total cases: 247
deterministic hits: 65
vlm reviews: 231
live vlm reviews: 166
remote vlm requests: 100
local vlm resolutions: 66
vlm cache hits: 0
skipped cases: 16
final failures: 32
```

先保留前两轮当时的观察：

- triage 阶段召回已经达到 100%，说明“该不该进入复核”的问题基本被解决。
- 213643 adaptive 更接近“远端 VLM 直接参与最终判定”的早期状态，远端请求 124，但 final recall 只有 60.10%。
- 221926 aggressive 不是项目本身变强，而是同一套 benchmark 逻辑在更激进命令参数下的表现：远端请求减少到 60，本地分流增加到 106，final recall 提升到 82.27%，但仍有 36 个 FN。

224421 的提升幅度明显更大：相比 213643，它有 71 个旧 FN 转为 TP，同时有 2 个旧 TP 转为 FN，净增 69 个 TP；相比 221926 aggressive，它还有 30 个旧 FN 转为 TP，同时有 6 个旧 TP 转为 FN，净增 24 个 TP。改善主要集中在 stage1 和 stage2：从 213643 到 224421，FN->TP 中 stage1 有 40 个、stage2 有 20 个、stage4 有 5 个、stage5 有 6 个。

这次提升不应简单解释成“远端 VLM 更准了”。从结果分布看，224421 的远端成功数反而少于 213643：
success 从 123 降到 41，**failed** 增加到 58；但 final recall 仍然大幅上升。更合理的解释是 31b7aad 改变了 final 判定和上下文使用方式：

- 引入 **risk_level**，并把 **content_exposed**、**in_progress**、**completed** 等风险阶段纳入 positive 口径，使 AI 输入框暴露敏感内容、屏幕共享、截图、VM/远程复制等“内容已暴露但未必有发送成功提示”的场景不再被压成 FN。
- VLM success 时不再只依赖 EventCorrelator 是否产出 **upload_candidates**，而是采用 **vlm_positive or upload_candidates**，因此模型已经给出高置信风险阶段时可以直接进入 final positive。
- EventCorrelator 不再拿空日志做关联，而是拿 **_logs_for_correlation(logs, fallback_meta)** 过滤后的上下文日志；这会让 visual verdict 与邻近日志、敏感文件、窗口事件重新绑定，减少“VLM 看

到了风险但关联层没有候选”的漏报。

- 本地分流从 42 增加到 66，其中 `local_ocr_positive` 从 16 增加到 34，说明 OCR 完成态/敏感名命中承担了更多兜底；这会提高召回，但也需要继续看误报边界。
- VLM 非 success 的 case 在 final 上更偏保守处理。224421 中 45 个从 213643 的 FN 转 TP 的 case 对应 `failed` 状态，这解释了为什么召回上升很快，也解释了 FP 从 8 增加到 20。

因此，224421 更像是“召回优先的 adaptive 口径修正”：它用风险阶段、本地证据和关联上下文把许多原本卡在 VLM/关联层之间的漏报补了回来；代价是正常外部应用场景更容易被保守判为 positive。

从三组结果看，后续优化重点应该是：

- 保留 224421 对内容暴露态、VM/远程、AI 输入、截图/共享等场景的召回提升。
- 单独压低 stage1 正常外部应用场景的 FP；从 213643 到 224421，TN->FP 有 12 个，其中 stage1 占 9 个。
- 把 VLM `failed` 的保守兜底拆得更细，避免所有复核失败都直接推高 final positive。

14. 测试覆盖

主要回归测试在 `tests/test_e2e_regressions.py`。

覆盖内容包括：

- prompt loader 不被同名模块污染。
- fixtures 不出现中文乱码。
- FrameAnalyzer 限制 VLM 帧数但保留上下文。
- Qwen VLM response 后处理、去重和噪声过滤。
- 曾经漏报的 violation case。
- VLM fallback gate 对 AI、普通聊天、VM、剪贴板、会议等场景的策略。
- realistic log fixtures 的策略矩阵。
- log-first 曾经漏掉的 case 变成 deterministic event。
- 可见敏感 anchor 能进入 VLM。
- 内容粘贴被视为 transfer candidate，而不是直接当日志外发。
- 派生文件上传能生成 Datalog leak path。
- EventCorrelator 能回填主检测结果。
- Python Datalog 引擎不会在循环传播中无限扩张。

运行方式：

```
python -m unittest tests.test_e2e_regressions
```

15. 方案演进

从 git 历史看，项目大致经历了四个阶段。

阶段一：视频/VLM 原型

早期重点是从录屏里找关键帧，用 OCR 和 VLM 判断用户行为。这个阶段解决了“纯日志看不见 UI 语义”的问题，但成本高，且容易因为抽帧不准漏掉短暂状态。

阶段二：log-first

后续加入 `LogFirstDetector` 和 NAS benchmark，把确定性日志检测前置。只要日志能说明敏感文件外发，就不调用 VLM。这一步显著降低了成本，也让错误更容易定位。

相关提交包括：

- `87f8aa7` Add offline detection benchmark
- `6110e6a` Improve log-first triage and NAS tooling
- `724455e` Add visual anchors for VLM triage
- `1fa8f84` Add live VLM verification to NAS benchmark

阶段三：EventCorrelator

项目随后引入 `02-EventCorrelator`，把日志、前台应用、窗口、VLM segment 和文件 lineage 统一成 `CorrelationBundle`。这一步让系统从“检测脚本集合”开始变成“证据链系统”。

关键提交：

- `2a31428` Add event correlator architecture layer
- `90e338c` Add benchmark table reporter
- `6baef0b` Update VLM model and improve recall routing

阶段四：VLM 成本和召回优化

最近的改动集中在 NAS live VLM：

- 自适应帧预算。
- segment 化 review window。
- OCR 预筛。
- 本地 gate。
- VLM verdict 缓存。
- adaptive/aggressive 模式对比。

关键提交：

- `3c4f64b` Add adaptive VLM frame budgeting
- `26ca26c` Reduce VLM image load with segments and OCR
- `7c8056f` Add selectable OCR engine for VLM prefilter
- `ad20ea3` Optimize NAS VLM review flow
- `31b7aad` Improve adaptive NAS VLM benchmark

16. 当前技术取舍

取舍	当前选择	原因
先看日志还是先看视频	先看日志	成本低、可解释、适合定位时间窗
是否全量调用 VLM	不全量调用	大量正常样例不值得远端复核
OCR 是否可直接定案	只在强条件下本地 positive	OCR 容易误识别，必须保留审计字段
文件派生怎么追	显式字段优先，stem 启发式补充	当前日志不是所有平台都有 inode/hash
Datalog 是否强依赖 Souffle	不强依赖	Windows 开发和运行需要 Python fallback
是否立即重构目录	暂不硬迁移	保持 benchmark、旧脚本和报告可运行

17. 后续改进方向

- 1. 固化数据集 manifest。
 - 现在部分类别、app、held-out 信息仍依赖路径或文件名推断。
 - 后续应显式维护 case metadata。
- 2. 增强 lineage 强特征。
 - 引入文件大小、hash、创建时间、inode/file id。
 - 减少同名文件 stem 推断误连。
- 3. 更细分 VLM verdict 状态。
 - preparation
 - selected_or_attached
 - content_exposed
 - in_progress
 - completed
 - none
- 4. 做类别级 FN/FP 分析。
 - 邮件、会议、VM、AI 内容暴露、压缩/转换是重点。
- 5. 落地图数据库。
 - 当前 evidence graph 主要存在 JSON bundle 中。
 - 如果后续确实需要跨会话查询，可以再增加 Neo4j writer；当前代码没有把证据图写入 Neo4j。
- 6. 收敛历史命名。

- 报告和部分代码里仍有 module3/module4、RiskHunter/ThreatDetector 等旧名。
 - 建议先稳定 `data_leak_detector.*` API，再迁移物理目录。
-

18. 一句话总结

DataLeakDetector 的本质是一个“证据链构造器”：日志负责找到敏感源和可疑窗口，OCR/VLM 负责补足 UI 语义，EventCorrelator 负责把零散证据绑定成候选泄露事件，LeakReasoner 负责用 Datalog 给出可解释的泄露路径。当前系统已经从视频大模型原型演进为日志优先、视觉兜底、符号推理收束的工程化检测链路。